
cTreeBalls

Release 1.0.0

Mario A. Rodriguez-Meza

May 19, 2026

CONTENTS:

1	Overview	1
1.1	Introduction	1
1.2	Compiling and getting started	1
1.3	Configuration	2
1.4	Parameters	2
1.5	Python	2
1.6	Plotting utilities	3
1.7	License	3
1.8	Acknowledgements	3
2	Installation	5
2.1	Makefile	5
2.2	Makefile settings	5
2.3	Makefile plataform machine	5
3	Parameters	7
3.1	Parameters related to the searching method	7
3.2	Parameters to control the I/O file(s)	7
3.3	Parameters to control histograms and their output files	8
3.4	Set of parameters needed to construct a test model	8
3.5	Miscellaneous parameters	8
4	Catalog files	11
4.1	Standard formats	11
4.2	Miscellaneous formats	11
5	Pre/Post processing	13
5.1	Pre-processing	13
5.2	Post-processing	13
6	Code structure	15
6.1	Runtime flow	15
6.2	Module map	16
6.3	Core state model	17
6.4	Tree and search lifecycle	17
6.5	Histogram ownership	18
6.6	Adding or changing parameters	18
6.7	Adding input/output formats	18
6.8	Adding search methods	19
6.9	Add-on sockets	19
6.10	Python wrapper notes	19

6.11	Generated and vendored code	19
7	AddOn's	21
7.1	Include files	21
7.2	Source files	21
8	Python interface	23
8.1	Basic lifecycle	23
8.2	Run levels	23
8.3	Available getters	24
8.4	Cython interface	24
8.5	See also	24
9	Hands on	25
10	2-point correlation functions	27
11	3-point correlation functions	29
11.1	Harmonic base	29
12	Indices and tables	31

OVERVIEW

cTreeBalls: Correlation functions computation with Tree/Balls methods

Author: Mario A. Rodriguez-Meza

For download and information, see <https://github.com/rodriguezmeza/cTreeBalls>

1.1 Introduction

Correlation function computation with Tree/Balls methods (short-name cBalls) is a C code for computing correlation functions using tree and balls methods. So far can compute 2 points correlation function (2pcf) and 3 points correlation function for scalar fields like weak lensing convergence.

1.2 Compiling and getting started

Download the code by cloning it from <https://github.com/rodriguezmeza/cTreeBalls>

Dependencies: cBalls optionally needs gsl version 2.7.1 and cfitsio version 4.4.1 installed in your system. Go to its web page [GSL](#) for details or ask to your system administrator. Make necessary changes in `Makefile_machine` file and look up for GSL. Can be switched off: `USEGSL = 0` in `Makefile_settings`. I/O cfitsio library is set it OFF. Can be set it ON in `addons/Makefile_settings`.

Go to the cTreeBalls directory (`cd cTreeBalls`) and compile (`make clean; make`). If the first compilation attempt fails, you may need to open the `Makefile_machine` file and adapt the name of the compiler (default: `gcc`), of the optimization flag (default: `-O4 -ffast-math`) and of the OpenMP flag (default: `-fopenmp`; this flag is facultative, you are free to compile without OpenMP if you don't want parallel execution; note that you need the version 4.2 or higher of `gcc` to be able to compile with `-fopenmp`). The code has been tested with `gcc` version 10 and would be working with version 11, 12. (In particular, for compiling on Mac ≥ 10.9 despite of the clang incompatibility with OpenMP).

To check that the code runs, type:

```
cd tests
../cballs parameters_explained
```

The `parameters_explained` file is a reference input file, containing and explaining the use of all possible input parameters.

By default **cBalls** reads/writes catalog of points to analyzed files in 4-column format with x, y, z columns first and then value of the convergence field. It has a two line header:

```
# nbody NDIM Lx Ly Lz
# nbody-value NDIM-value Lx Ly Lz - values
```

Try running:

```
../cballs nbody=6480 o=points_on_sphere testmodel=unit-sphere-random options=stop
```

In the Output directory you will have a file: `points_on_sphere.txt`. View its contents to see the two lines header and the 4 columns structure of data.

Note: in the above example `points_on_sphere` was not given an extension. By default cBalls gives to the output files the extension `.txt`.

On top of that, if you wish to modify the code, you will find comments directly in the files in the folder `addons`, and the modules you may add must go in this folder. See one of the most simple in `addons/direct_method`.

For the moment you may consult man page:

```
man ../docs/cballs.m
```

or open with a browser the html file: `docs/man/cballs.html`

See more details about parameters needed by cBalls below (*Parameters*).

1.3 Configuration

cBalls can be configured by switching on/off several options. Configuration file is `Makefile_setting`. Search method `balls-omp` in folder `addons/balls_omp`: `Makefile_settings_balls_omp` has other options that can be switched on/off.

1.4 Parameters

The list of available command line parameters can be consulted using the `--help` flag:

```
../cballs --help
```

or its short version:

```
../cballs -h
```

1.5 Python

cBalls python module (`cbalys`) will be installed if you use:

```
make clean; make all
```

Note: if problems, you can go to python directory and execute:

```
python setup.py build
python setup.py install --user
```

To test it go to directory `tests` and 'run':

```
python test_cython_balls.py
```

Note: this interface in Cython was tested in a python environment with python3.7.

1.6 Plotting utilities

Several Jupyter notebooks, written by Abraham Arvizu and Eladio Moreno, are available to process cBalls results. They are in the github repository:

https://github.com/joar-cafe/CBalls_plots/tree/main/benchmarks

or you can find more scripts in `tests` folder.

1.7 License

cBalls is written by Mario A. Rodriguez-Meza, and is distributed under the [MIT license](#). If you use this program in research work that results in publications, please cite the following paper:

Abraham Arvizu et al., [arXiv:2048.16847](#)

1.8 Acknowledgements

cBalls use/is based on the following codes or projects:

- [Zeno](#)
- [Gadget-2](#)
- [CUTE](#)
- [Numerical recipies](#)
- [GSL](#)
- [CLASS](#)
- [CFITSIO](#)
- [HEALPix](#)

Also author acknowledges for helpful discussion and testing to the following people:

- Abraham Arvizu
- Alejandro Aviles
- Juan Carlos Hidalgo
- Eladio Moreno
- Gustavo Niz
- Axel Romero Tisnado
- Sofia Samario

INSTALLATION

This section describes the installation settings of cBalls:

2.1 Makefile

2.2 Makefile settings

2.3 Makefile plataform machine

PARAMETERS

This section describes the various parameters cBalls needs for controlling what the searching process do:

3.1 Parameters related to the searching method

searchMethod

(str or list) [alias: search] The searching method to use. Default is `tree-omp-sincos`. Fastest method so far is `octree-ggg-omp`.

Use it as:

```
searchMethod = octree-ggg-omp
```

In command line version do not use spaces before and after = or it won't be parsed correctly. In a parameter file you have more liberty.

mChebyshev

(positive int) [alias: mcheb] The number of multipoles to compute for a 3pcf computation. Number of them are: `mChebyshev + 1`, because it includes the monopole.

Use it as:

```
mChebyshev = 7
```

Default value is 7. As multipoles comes from a harmonic expansion, we may be interested in computing the whole 3pcf. This process involves a FFT, therefore give `mChebyshev + 1` as a power of 2.

3.2 Parameters to control the I/O file(s)

infile

(str, default="") [alias: in] File names with points to analyse.

infileformat

(str, default=columns-ascii) [alias: infmt] Data input files format (columns-ascii, binary or takahasi)

The input columns for `columns-ascii` format:

- `x` = x position of a point in the catalog.
- `y` = y position of a point in the catalog.
- `z` = z position of a point in the catalog.

- `kappa` = The kappa value of the point.

3.3 Parameters to control histograms and their output files

useLogHist

(bool, default=true) Which type of binning should be used.

logHistBinsPD

(float) The minimum separation to include in the histograms.

rangeN

(float) The maximum separation to include in the histograms.

rminHist

(float) The minimum separation to include in the histograms.

sizeHistN

(int) The number of output bins to use.

3.4 Set of parameters needed to construct a test model

seed

(int, default=123) Random number seed to test run or useful to change a random region in Takahasi simulations.

testmodel

(str, default=simple-cubic-random) [alias: tstmodel] Test model name to analyse.

nbody

(int, default=16348) Number of points to test.

lengthBox

(float, default=10000) [alias: lbox] Length of the box to test.

3.5 Miscellaneous parameters

script

(str, default=("")) Scripts in shell or python that can be run in pre-processing or post-processing.

stepState

(int, default=10000) number of steps to save a state-run info (pivot number completed in the log file).

verbose

(int, default=1) [alias: verb] How verbose the code should be during processing.

- 0 = no output unless there is an error
- 1 = output warnings
- 2 = output progress information
- 3 = output extra debugging lines

verbose_log

(int, default=1) [alias: verblog] To print messages to a log file cballs.log in directory tmp under output directory given by the parameter: `rootDir``.

Amount of message information is controlled by the int given.

numberThreads

(int, default=4) [alias: nthreads] How many OpenMP threads should be used.

It is needed to switch on OpenMP: `OPENMP_MACHINE = 1` in `Makefile_settings` and recompile cBalls again.

options

(str, default="") [alias: opt] You may give here various code behavior options.

Use it as:

```
options = str1,str2,str3,...
```

where str# is one of the:

- stop = stop execution before searching process
- compute-HistN = compute NN encounters and save histogram in a file
- and-CF = if you use `compute-HistN` then you may compute and save the correlation function of NN encounters (the equivalent to the radial distribution function in liquids).
- no-one-ball = during the searching process does not use balls criterion to speed up the code

Note

- It is not necessary to specify all the parameters. You need to give only the ones appropriate to the run. The rest of parameters will use their default values if they are OK with you.
- When you specify the root output directory using: `rootDir`, and this is a single directory that will be located in the pwd dir, then do not use `./` at the beginning of the name or `/` at its end.

CATALOG FILES

This section describes the various formats of catalog points you need to compute correlation functions on them. They are given with the option: `infileformat`.

4.1 Standard formats

columns-ascii

The file(s) with the data to be correlated.

binary

The file with data to be correlated in binary format.

takahashi

The file with data to be correlated using Takahashi et al simulations format (can be transformed to Healpix format).

4.2 Miscellaneous formats

gadget

Gadget files can be read using this format. Catalog can be only one file or composed of several files.

multi-columns-ascii

This format is to be able to read any ascii file with values arranged in columns. Positions can be in any order and with/without header. For example, files with halo catalogs from Rockstar can be read.

fits-healpix

to read maps in Healpix format using `cfitsio` library. This format permits to apply a mask in the same format.

PRE/POST PROCESSING

This section describes the various pre/pos processing for controlling what the *cballs* before/after main processing:

5.1 Pre-processing

5.2 Post-processing

CODE STRUCTURE

This page is a developer-oriented map of the cTreeBalls source tree. It is intended to answer two questions before editing the code: where does a run move through the program, and which files own each piece of state.

6.1 Runtime flow

The command-line executable follows this high-level sequence:

1. `main/main.c` creates the run state objects and calls `StartRun`.
2. `source/startrun.c` reads parameters, checks them, initializes random generators, prepares output paths, and allocates the run-owned arrays.
3. `source/cballs.c` enters `MainLoop` and dispatches to `EvalHist`.
4. `source/treeload.c` builds one tree per selected input catalog with `MakeTree`.
5. `source/search.c` walks the tree, accepts or rejects node pairs, and accumulates thread-local histogram contributions.
6. `source/cballsutils.c` supplies shared search utilities, histogram allocation and normalization helpers, geometry transforms, and acceptance predicates.
7. `source/cballs.c` writes evaluated histograms through its `PrintHist*` helpers.
8. `source/cballsio.c` closes files and releases memory in `EndRun`.

The Cython wrapper in `python/cballys.pyx` calls the same C stages, but it splits them into a dependency-checked `Run(level=[...])` workflow so Python callers can compute and then read arrays back without going through the command line.

6.2 Module map

Path	Responsibility
main/main.c	CLI entry point. Creates struct <code>cmdline_data</code> and struct <code>global_data</code> , records start times, reads the parameter file argument when <code>GETPARAM</code> is disabled, and runs <code>StartRun</code> , <code>MainLoop</code> , then <code>EndRun</code> .
include/globaldefs.h	Central include file. Pulls in common libraries, declares global tables such as <code>bodytable</code> , <code>roottable</code> , and <code>nodetablescalelev</code> , selects <code>GSL</code> or <code>Numerical Recipes</code> support, and exposes project-wide error and debug macros.
include/cmdline_defs.h	Parameter registry used by the <code>getparam</code> interface. Add command-line parameters here first, then wire them through <code>startrun.c</code> .
include/cmdline_data.h	Parsed user options and derived command state. This is the user-facing configuration object passed through almost every major routine.
include/global_data.h	Run state that is derived, allocated, or accumulated during execution: file paths, histogram arrays, tree statistics, timing counters, input catalog metadata, and memory ownership flags.
include/datastruc_defs.h	Body, cell, and node layout. <code>body</code> and <code>cell</code> both embed a common <code>node</code> header so tree traversal can work on <code>nodeptr</code> values.
include/datastruc_hist.h	Per-thread and global histogram workspaces used by the search code.
include/protodefs.h	Shared function prototypes and add-on include hook for APIs that need to be visible across C modules.
source/startrun.c	Startup lifecycle: command-line and parameter-file parsing, <code>StartRun_Common</code> , validation, data loading or test-data generation, output setup, and memory allocation.
source/cballsio.c	Input/output formats, output directory and filename setup, writing catalogs, and all teardown routines called from C and Cython.
source/testdata.c	Synthetic catalog generation for simple cubic and unit-sphere runs.
source/treeload.c	Octree construction, root-cell setup, body insertion, cell properties, tree threading, scan-level tables, and tree cleanup.
source/search.c	The default <code>octree-sincos-omp</code> search. Owns the OpenMP pivot loop, tree walk, node summation, and merging of thread-local histograms.
source/cballsutils.c	Shared search utilities: histogram initialization, body property reduction, acceptance/rejection checks, periodic wrapping, coordinate conversion, and histogram diagnostics.
source/cballsutils_with-RADECPOC.c	RA/DEC-position variant of <code>cballsutils.c</code> . Keep it in sync with <code>cballsutils.c</code> when enabling or maintaining that build path.
source/cballs.c	Main computation driver and histogram output dispatch. This is where search-method selection reaches the concrete search implementation.
python/cballsys.pyx	Python class wrapper around the C lifecycle. Converts a Python parameter dictionary into CLASS-style <code>file_content</code> , calls C modules, exposes histogram getters, and releases C allocations.
python/ccballsys.pxd	Cython declarations for the C structs and functions that Python needs. Only expose fields here when the wrapper really consumes them.
addons/	Optional search methods, input formats, and bundled libraries. Add-ons connect through the <code>//B socket</code> : include points in core files.

6.3 Core state model

Most C routines receive two pointers:

struct cmdline_data *cmd

Values requested by the user or read from a parameter file. Treat this as configuration. It contains items such as searchMethod, histogram sizes, input/output names, verbosity, and option strings.

struct global_data *gd

Values computed during the run. Treat this as mutable execution state. It owns histogram buffers, input catalog metadata, output paths, timing counters, tree statistics, and allocation flags used during cleanup.

The large catalog and tree tables are declared as project globals in include/globaldefs.h:

bodytable[MAXITEMS]

Per-catalog body arrays.

roottable[MAXITEMS]

One tree root per loaded catalog.

nodetablescalev[MAXITEMS] and nodetablescalev_root[MAXITEMS]

Flattened node tables used by scan-level traversal and diagnostics.

The body/cell hierarchy is defined in include/datastruc_defs.h. A body contains a node named `bodynode` and a body id. A cell contains a node named `cellnode`, a size, a linked `more` pointer, and `NSUB` subcells. The macros in that header, for example `Pos(x)`, `Mass(x)`, `Radius(x)`, and `Subp(x)`, are the intended field access style throughout the C code.

6.4 Tree and search lifecycle

`MakeTree` in `source/treeload.c` starts with `newtree` and a root cell, then loads bodies into subcells using `loadbody` and `subindex`. After insertion, `hackcellprop` computes cell-level properties used by the search, `threadtree` links traversal order, and `scanLevel` prepares node tables for level-based operations.

`searchcalc_normal_sincos` in `source/search.c` is the default search path for `searchMethod=tree-omp-sincos/octree-sincos-omp` style runs. It:

1. Initializes global and per-thread histogram workspaces.
2. Runs an OpenMP loop over pivot bodies from the selected pivot catalog.
3. Walks the scanning catalog tree with `normal_walktree_sincos`.
4. Uses acceptance/rejection helpers from `source/cballsutils.c` to decide whether a body/cell or cell/cell pair contributes.
5. Accumulates 2PCF and, when enabled at compile time, 3PCF Chebyshev/sine/cosine components.
6. Merges thread-local histograms back into `gd`.

6.5 Histogram ownership

Histogram buffers are allocated during startup and search initialization, then freed through the `EndRun_FreeMemory_*` routines in `source/cballsio.c`. The important ownership rule is: if a new allocation is attached to `cmd` or `gd`, it needs a matching allocation flag or an existing cleanup stage that cannot miss it. This is especially important for the Python wrapper, because `cballs.struct_cleanup` calls the granular C cleanup routines instead of only exiting the process.

Common histogram outputs include:

histNN

Neighbor count histogram.

histCF

Correlation-function style output derived from counts.

histXi2pcf

Two-point correlation function output.

histZetaM* and mhistZetaM*

Three-point/Chebyshev multipole outputs, split by sine/cosine component and multipole index when the corresponding compile-time options are enabled.

6.6 Adding or changing parameters

To add a user-visible parameter:

1. Add the default, description, and optional short name in `include/cmdline_defs.h`.
2. Add fields to `struct cmdline_data` in `include/cmdline_data.h` if the value needs to persist.
3. Read the parameter in `ReadParametersCmdline` and `ReadParameterFile` in `source/startrun.c`.
4. Print it from `PrintParameterFile` so generated parameter snapshots remain reproducible.
5. Validate derived constraints in `CheckParameters` when invalid combinations would otherwise fail later.
6. If the parameter is exposed to Python, mirror it in `python/ccballys.pxd` and set or document it in `python/cballys.pyx`.

6.7 Adding input/output formats

Input format names are converted to integer tags by `infilefmt_string_to_int` in `source/cballsio.c`. Add a new tag in the same style as the existing formats, implement the parser near the other `inputdata_*` functions, and add the switch case in `InputData` or `InputData_all_in_one`.

Output formats follow the matching pattern through `outfilefmt_string_to_int`, `outputdata`, and the `outputdata_*` helpers. Keep filename construction in `setFilesDirs/setFilesDirs_log` so command line, tests, and Python runs all agree on output paths.

6.8 Adding search methods

Search-method selection happens in two places:

1. `search_method_string_to_int` in `source/startrun.c` maps user strings to integer constants.
2. `EvalHist` in `source/cballs.c` switches on `gd->searchMethod_int` and calls the concrete implementation.

New search methods usually need prototypes in `include/protodefs.h`, compile settings in the Makefiles or add-on make fragments, and output handling in `source/cballs.c` only if they produce different histogram families.

6.9 Add-on sockets

Core files contain `//B socket:` blocks guarded by `#ifdef ADDONS`. These blocks include generated or configured files such as `*_include.h` so optional modules can add fields, prototypes, switch cases, and setup logic without editing every core file manually. When adding an add-on, keep its ownership localized under `addons/<name>/` and document which socket files it expects to be copied or generated.

6.10 Python wrapper notes

The wrapper class `cballs` in `python/cballys.pyx` keeps a Python parameter dictionary in `self._pars`. `Run` converts that dictionary into `file_content`, calls `input_read_from_file`, then advances through `StartRun_Common`, `PrintParameterFile`, `SetNumberThreads`, and `MainLoop` according to the requested dependency level.

Getter methods such as `getrBins`, `getHistNN`, `getHistXi2pcf`, and `getHistZetaMsincos` copy C arrays into NumPy arrays. They assume the relevant run stage has already completed and that `sizeHistN` still matches the allocated histogram dimensions.

When changing C struct fields consumed by Python, update both `python/ccballys.pxd` and the getter or setup code in `python/cballys.pyx`. The wrapper intentionally exposes only a subset of the C structs; avoid adding fields to the `.pxd` unless Python needs direct access.

6.11 Generated and vendored code

The repository includes generated Sphinx output under `docs/_build` and large vendored libraries under `addons/`. Prefer editing the source `.rst` files, core `source/` and `include/` code, or add-on source fragments rather than generated HTML/LaTeX or third-party library internals.

ADDON'S

Add-ons extend cBalls with optional search methods, catalog formats, and supporting libraries without making every core file permanently depend on those features. They are enabled at compile time through `Makefile_settings` and the add-on make fragments under `addons/`.

7.1 Include files

Core headers and modules contain guarded `//B socket:` blocks such as:

```
#ifdef ADDONS
#include "protodefs_include.h"
#endif
```

An add-on uses these include sockets to add prototypes, struct fields, parameter definitions, switch cases, and implementation hooks. Keep add-on specific declarations in the add-on include files rather than editing unrelated core code.

Common socket targets include:

`cmdline_defs_include.h`

Adds new command-line parameters.

`protodefs_include.h`

Adds prototypes that must be visible across C modules.

`globaldefs_include_*.h` and `global_data_include_*.h`

Adds globals or run-state fields required by the add-on.

`cballs_include_*.h`, `startrun_include_*.h`, `cballsio_include_*.h`

Adds search-method dispatch, startup, and I/O cases.

7.2 Source files

Each add-on should keep its implementation and build fragment in its own subdirectory, for example `addons/cfitsio`, `addons/gadget_io`, `addons/kdtree_box_omp`, or `addons/octree_box_omp`. The add-on `Readme.txt` and `To_add_in_Makefile_addons` files explain which fragments must be included in the top-level add-on build.

When adding a new search add-on, wire it through the same conceptual path as a core search method:

1. Define the command-line search string and integer method tag.
2. Add parameter parsing or validation hooks if the method needs extra options.

3. Add the method's prototype through the prototype socket.
4. Add the EvalHist switch case through the cballs socket.
5. Make sure any method-specific allocations are released by an existing or new cleanup hook.

When adding a new I/O add-on, add the format string conversion, the parser or writer implementation, and the InputData/OutputData switch case through the I/O sockets.

PYTHON INTERFACE

The Python extension is implemented by `python/cballys.pyx` and declared to Cython through `python/ccballys.pxd`. It wraps the same C lifecycle used by the `cballs` executable, but exposes it as a Python class named `cballs`.

8.1 Basic lifecycle

Typical Python use follows this pattern:

```
from cballys import cballs

cb = cballs()
cb.set(
    infile="catalog.txt",
    infileformat="columns-ascii",
    rootDir="Output",
    options="compute-HistN",
)
cb.Run()
rbins = cb.getrBins()
xi2 = cb.getHistXi2pcf()
```

`set` updates the internal parameter dictionary. `Run` converts that dictionary into the C `file_content` representation, calls the startup routines, executes the main search, and leaves C-owned histograms available for getter methods. `clean_all` releases C allocations and clears the Python parameter dictionary.

8.2 Run levels

`Run(level=[...])` accepts the name of the last C stage the caller needs. The wrapper expands dependencies automatically. The default level is `["MainLoop"]`, which performs a full calculation. Internally the common stage order is:

1. `input` reads Python parameters into C structures.
2. `StartRun_Common` validates parameters, initializes state, loads input catalogs or test data, and allocates memory.
3. `PrintParameterFile` writes a reproducible parameter snapshot.
4. `SetNumberThreads` applies the OpenMP thread count.
5. `MainLoop` builds trees, runs the selected search, and evaluates histograms.

8.3 Available getters

The wrapper exposes small, NumPy-oriented getters for the C arrays used most often from notebooks and scripts:

getrBins

Radial bin centers or edges as prepared by the C run.

getHistNN and getHistCF

Neighbor counts and correlation-function outputs.

getHistXi2pcf

Two-point correlation function array.

getHistZetaMsincos(m, type) and getHistZetaM_EE(m)

Two-dimensional 3PCF/multipole matrices for the requested multipole index.

getNThreads, getTheta, getrsmooth, getCPUTime, getVersion, getRootDir, and allocation-flag getters expose selected scalar state from the underlying C structs.

8.4 Cython interface

python/ccballys.pxd intentionally declares only the fields and C functions needed by the wrapper. When a C field is renamed, moved, or newly needed by Python, update the .pxd declaration and the matching logic in python/cballys.pyx together.

Memory is owned by the C code after StartRun_Common. The wrapper tracks C allocation flags and calls the granular EndRun_FreeMemory_* routines from struct_cleanup. New C allocations that should survive long enough for Python getters must also be reachable by the cleanup path.

8.5 See also

For a broader map of the C modules and Python wrapper, see [Code structure](#).

HANDS ON

This section describes a practical guide to use **cBalls**:

Let us test one of the [Takahashi](#) realizations. We download the realization using unix `wget` command in the terminal:

```
wget http://cosmo.phys.hirosaki-u.ac.jp/takahasi/allsky_raytracing/sub1/nres12/allskymap_  
↪nres12r000.zs9.mag.dat
```

Then we will have in our working directory the file `allskymap_nres12r000.zs9.mag.dat`, 3 Gb in size with ~200 million points distributed on the surface of a unit sphere.

2-POINT CORRELATION FUNCTIONS

cBalls can compute 2-point correlations (2pcf) when counts (N) or a scalar field are involved (like convergence K):

NN

It is the normal 2-point correlation function of number counts (typically galaxy counts).

KK

So far we have consider that the scalar field is the convergence in weak lensing κ . Then this gives the 2-point kappa-kappa correlation function.

N represent simple counting and K represent a real scalar field, like convergence in weak lensing.

3-POINT CORRELATION FUNCTIONS

cBalls can also compute 3-point correlations (3pcf) when counts (N) or a scalar field are involved (like convergence κ). In particular to compute the correlation $\mathcal{K}$ the numerical code use the harmonic base.

11.1 Harmonic base

INDICES AND TABLES

- `genindex`
- `modindex`
- `search`